Statistical Programming Languages

Programming

Katarzyna Reluga[1]

[1]Acknowledgements: Thanks to Manuel Pfeuffer, Maarten Jung, and Tobias Wistub for sharing notes from earlier editions.

Functions

- Functions implement tasks you want to do repeatedly
- Help modularising your code

```
myfun <- function(x) {
  # The return statement terminates the function
  # and returns the value inside '()'
  return(x * x)
}

# Calling the function
myfun(2)     # Without naming the parameter
## [1] 4
myfun(x = 2) # With naming the parameter
## [1] 4
```

Functions

- Functions may take several parameters
- The different parameters are separated by comma

```
myfun <- function(x, a) {
  r <- a * sin(x)
  return(r)
}
myfun(pi / 2, 2)
## [1] 2
# When naming and calling parameters, the order does not matter
myfun(x = pi / 2, a = 2)
## [1] 2
myfun(a = 2, x = pi / 2)
## [1] 2
myfun1 <- function(x, a) a * sin(x); # short
myfun1(pi / 2, 2)
## [1] 2
```

Functions

- Functions may have optional parameters
- Optional parameters have default values
- We can call the function with or without the optional parameter

```
# a is optional with default 1
myfun2 <- function(x, a = 1) {
  a * sin(x)
}

myfun2(pi / 2, 2) # Provide a different value a
## [1] 2
myfun2(pi / 2)    # Use the default value a
## [1] 1
```

Functions

- A function can return any R data structure
  - Scalars, vectors, matrices
  - Data frames
  - Arrays, lists, etc.

```
myfun3 <- function(x, a = 1) {
  r1 <- a * sin(x)
  r2 <- a * cos(x)
  # Return a list containing two values
  return(list(r1 = r1, r2 = r2))
}

myfun3(pi / 2)
## $r1
## [1] 1
##
## $r2
## [1] 6.123234e-17
```

Functions

Calling the same function in different ways

```
myfun4 <- function(x, a = 1, b = 2) {
  r1 <- a * sin(x)
  r2 <- b * cos(x)

  return(c(r1, r2))
}

myfun4(pi / 2)         # a = 1, b = 2 (defaults)
## [1] 1.000000e+00 1.224647e-16
myfun4(pi / 2, 1, 2)   # a = 1, b = 2 (explicitly)
## [1] 1.000000e+00 1.224647e-16
myfun4(pi / 2, 2)      # a = 2, b = 2 (only a given)
## [1] 2.000000e+00 1.224647e-16
myfun4(pi / 2, a = 2) # a = 2, b = 2 (only a given)
## [1] 2.000000e+00 1.224647e-16
myfun4(pi / 2, b = 3) # a = 1, b = 3 (only b given)
## [1] 1.00000e+00 1.83697e-16
```

Pure Functions

- Functions should not depend on variables defined outside of it
- Functions should only provide an output but not change anything unintentionally (no side effects)

```
# How not to do it
y <- 1:10
my_plot <- function(x) {
  par(mfrow = c(1, 2)) # changes settings of par!
  plot(x, y)           # uses y!
  plot(y, x)
}
```

Pure Functions

- `on.exit(expr)` runs `expr` right before the function finishes—whether it ends normally, returns early, or errors.
- For multiple `on.exit()` calls, use `add = TRUE`.

```r
# How to do it better
my_plot <- function(x, y) {
  par_original <- par(no.readonly = TRUE) # save graphics settings
  on.exit(par(par_original))              # restore them on exit
  par(mfrow = c(1, 2))                    # make a 1x2 plotting layout
  plot(x, y)                              # first plot
  plot(y, x)                              # second plot
}
```

Set Operations

We can perform set operations on vectors

```
a <- 1:3
b <- 2:6
a %in% b        # Elements of a in b
## [1] FALSE  TRUE  TRUE
union(a, b)     # Create a union of a and b
## [1] 1 2 3 4 5 6
intersect(a, b) # Intersect the two vectors
## [1] 2 3
setdiff(a, b)   # Elements not in both vectors
## [1] 1
```

Control Flow: `if...else` Statements

Check whether a specific condition applies

```
# Simple if-statement
x <- 1
if (x == 2) {
  print("x = 2")
}

# if-else statement
x <- 1
if (x == 2) {
  print("x = 2")
} else {
  print("x != 2")
}
## [1] "x != 2"
```

Control Flow: `if...else` Statements

```
# Simple else-if-statement
x <- 1
if (x == 3) {
  print("x = 3")
} else if (x == 2) {
  print("x = 2")
} else {
  print("x != 3 & x != 2")
}
## [1] "x != 3 & x != 2"
```

Control Flow: `ifelse`

- Function that returns a value depending on a condition
- Works like `if` in MS Excel

```
# Simple example
x <- 1
ifelse(x >= 0, x, abs(x))
## [1] 1

# Catch negative values for sqrt()
x <- seq(2, -1)
sqrt(ifelse(x >= 0, x, NA))
## [1] 1.414214 1.000000 0.000000       NA
```

Control Flow: `switch`

- Switch avoids long `if...else` constructs
- Switch takes a numeric value or string as expression (i.e., type), and checks it against different cases

```
rootsquare <- function(x, type) {
  switch(type, square = x * x, root = sqrt(x))
}

rootsquare(10, 1)
## [1] 100
rootsquare(10, 2)
## [1] 3.162278
```

Control Flow: `switch`

If `type` is a string, R matches the string

```
centre <- function(x, type) {
  switch(type, mean = mean(x), median = median(x),
         trimmed = mean(x, trim = .1))
}

x <- rnorm(10)
centre(x, "mean")
## [1] 0.05858522
centre(x, "median")
## [1] -0.1694245
centre(x, "trimmed")
## [1] 0.009313125
```

Control Flow: `for` Loops

Often you will want to call a function iteratively for different values

```
for (i in 1:4) { print(i) }
## [1] 1
## [1] 2
## [1] 3
## [1] 4
for (i in letters[1:4]) { print(i) }
## [1] "a"
## [1] "b"
## [1] "c"
## [1] "d"

# generate a vector containing zeros of length 400
a <- numeric(400)
# fill a with 1:400
for (i in 1:400) {
  a[i] <- i
}
# takes much longer than a <- 1:400
```

Control Flow: `while` Loops

- `while` loops allow to perform an operation until a logical operation evaluates to `TRUE`
- If you make a mistake you get an **infinite** loop

```
i <- 0
while (i < 4) {
  i <- i + 1  # Don't forget this!
  print(i)
}
## [1] 1
## [1] 2
## [1] 3
## [1] 4
```

Control Flow: `repeat` Loops

- `repeat` starts an **unconditional** loop: it does **not** check a condition for you.
  You must test a condition **inside** the loop and `break` explicitly to exit (unlike `while`, which checks before each iteration).

```
i <- 0
repeat {
  i <- i + 1
  print(i)
  if (i == 4) {
    print('We are done!')
    break # Break exits a loop
  }
}
## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] "We are done!"
```

The `apply()` Function Family

- Set of highly optimized functions for iterative computations
- Much faster than standard loops
- Various versions for matrices, vectors, lists, . . . exist

`apply()`	Function used on matrix
`lapply()`	On lists and vectors; returns a list
`sapply()`	Like `lapply()`, returns vector/matrix
`mapply()`	Multivariate `sapply()`
`tapply()`	Table grouped by factors

apply()

- Apply a function sequentially to the rows/columns of a matrix
- Returns a vector

```
# 2 x 5 matrix
a <- matrix(1:10, nrow = 2)
a
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    3    5    7    9
## [2,]    2    4    6    8   10

# By row
apply(a, MARGIN = 1, FUN = mean)
## [1] 5 6

# By column
apply(a, MARGIN = 2, FUN = mean)
## [1] 1.5 3.5 5.5 7.5 9.5
```

lapply()

- Apply a function sequentially to a vector or list
- Returns a list

```
a <- rnorm(10)
b <- rnorm(2000)
c <- list(a, b)

# Apply the mean function to each element
# of the list c
lapply(c, mean)
## [[1]]
## [1] -0.5264933
##
## [[2]]
## [1] -0.03735574
```

sapply()

- Apply a function sequentially to a vector or list
- Returns a vector or matrix

```
a <- matrix(2:7, nrow = 2)
b <- matrix(1:6, nrow = 2)
c <- list(a, b)

sapply(c, as.vector) # Returns a matrix
##      [,1] [,2]
## [1,]    2    1
## [2,]    3    2
## [3,]    4    3
## [4,]    5    4
## [5,]    6    5
## [6,]    7    6
sapply(c, mean) # Returns a matrix
## [1] 4.5 3.5
```

mapply()

- Multivariate version of `sapply()`
- Applies FUN to the first, second, third, ... elements of the argument vectors

```
a <- c(1, 2)
b <- c(-1, -2)

mapply('+', a, b) # Add up the pairwise elements of a and b
## [1] 0 0
```

tapply()

Apply a function to the elements of a vector, grouped by factor

```
data(iris)
# Apply function mean to Sepal.Width by Species
tapply(X = iris$Sepal.Width, INDEX = iris$Species, FUN = mean)
##     setosa versicolor  virginica
##      3.428      2.770      2.974
```

Taking Time

`system.time()` helps tracking the performance of computations

```
x <- numeric(50000)
system.time(for (i in 1:50000) {
  x[i] <- rnorm(1)
})
##    user  system elapsed
##   0.017   0.000   0.016

system.time(rnorm(50000))
##    user  system elapsed
##       0       0       0
```

Making Code Faster

- Use functions of the `apply`-family instead of loops whenever possible
  - If you use a loop, create the object (e.g. a vector, matrix, ...) in advance and fill it inside the loop
  - Never use `cbind()` or `rbind()` inside a loop
- Parallelization to distribute CPU time to several cores
  - Code that can be expressed as a loop with **independent** iterations can be parallelized
  - Package `parallel` (pre-installed in `R`)
- Helpful packages for profiling and microbenchmarking are `profvis` and `bench`

Debugging

- When writing more complicated functions (with nested function calls) it can be hard to find the part responsible for an error
- Function `traceback()` can be called (after an error occurred)
- `options(error = recover)` opens an interactive debugger whenever an error occurs; When finished: Set options back to default with `options(error = NULL)`
- `browser()` opens the interactive debugger at the respective position in the source coude
- `debug()`, `debugonce()` insert the `browser()` function in the first line of the source code of the respective function; `undebug()` removes it

Debugging

```
options(error = recover)
f <- function(x) {
  return(log(x))
}
g <- function(n) {
  return(letters[n])
}

f(g(1))
## Error in log(x): non-numeric argument to
## mathematical function

Enter a frame number, or 0 to exit

1: f(g(1))

Selection: 0 # 0 exits, 1 opens debugging environment

options(error = NULL)
```